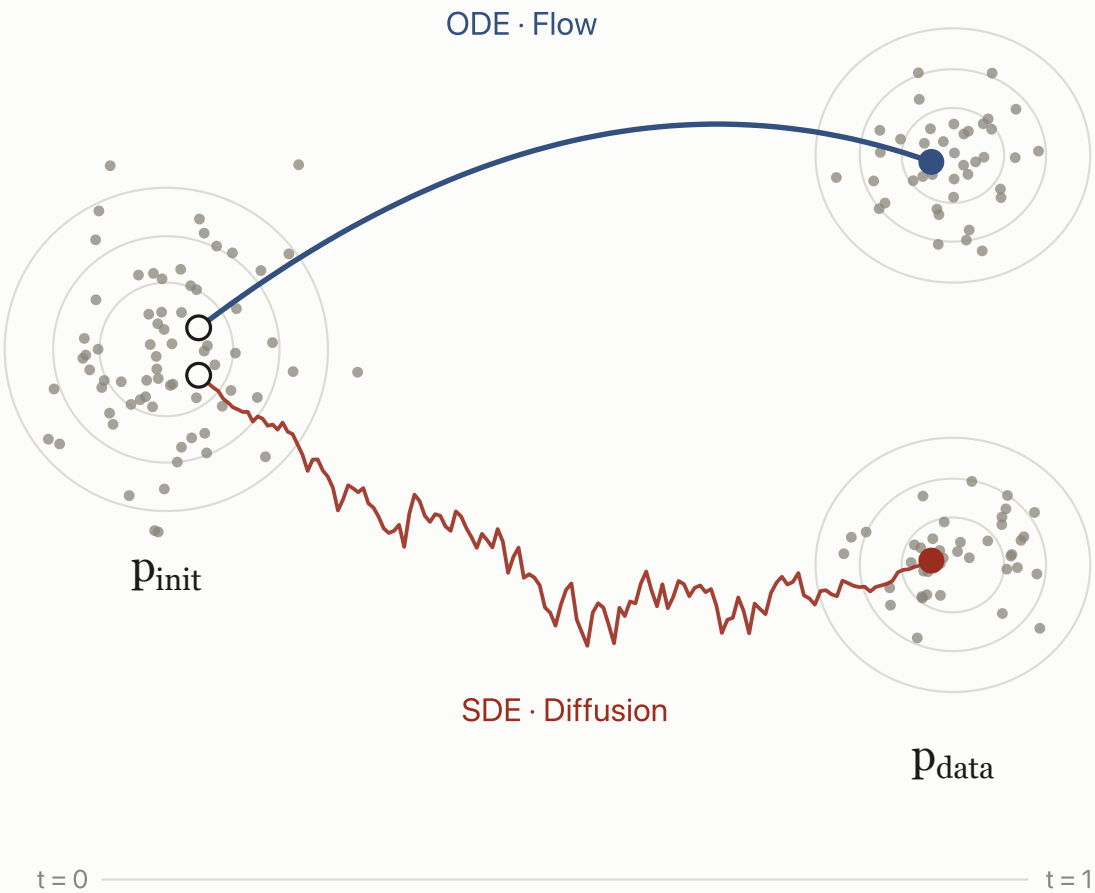
FLOW MATCHING & DIFFUSION · NOTES 01

从噪声到数据

一条平滑的流，一条带噪的扩散



目录

01 生成与采样

1.1 对象即向量

1.2 生成即采样

1.3 引导式（条件）生成

1.4 生成模型

02 Flow 模型：用 ODE 生成

2.1 ODE 与向量场

2.2 流（Flow）

2.3 常微分方程的数值模拟

2.4 流模型

03 Diffusion 模型：用 SDE 生成

3.1 随机微分方程（SDE）

3.2 布朗运动（Brownian Motion）

3.3 从 ODE 到 SDE

3.4 奥恩斯坦-乌伦贝克过程

3.5 SDE 的数值模拟

3.6 扩散模型

从生成到采样：Flow 与 Diffusion 模型

读完能写出两类模型的采样算法，并说清扩散系数 σ 的角色

课程： MIT 6.S184 Flow Matching & Diffusion Models　类型： 个人学习笔记　日期： 2026-10

生成一张图究竟是在做什么？本讲先把“生成”翻译成“从数据分布中采样”，然后给出两种把简单噪声变成复杂数据的动力学：沿确定性的常微分方程流动的 Flow 模型，与在此基础上叠加布朗运动噪声的 Diffusion 模型。两者的核心都是同一件事——用神经网络参数化一个向量场。

三句话读懂本讲

1. 生成即采样：生成一个对象，就是从数据分布中采样；生成模型是把一个易采样的初始分布变成数据分布的算法。
2. Flow 模型用 ODE：轨迹是确定的，随机性只来自初值；Diffusion 模型用 SDE：每一步还叠加布朗运动带来的噪声。
3. ODE 就是扩散系数 σ = 0 的 SDE，所以 Flow 是 Diffusion 的特例；两者的采样分别用欧拉法与欧拉-丸山法做数值模拟。

01. 生成与采样

在谈 Flow 与 Diffusion 之前，先要回答一个更朴素的问题：当我们说“生成一张图”时，数学上到底在做什么？本节分四步把这件事说清楚——先把对象变成向量，再把“生成”变成“采样”，然后推广到带条件的生成，最后给出生成模型的抽象定义。

1.1. 对象即向量

我们一般会把图像、视频表示为向量：

- 图像：考虑分辨率为 $H \times W$ 像素的图像，其中 H 表示图像的高度，W 表示宽度，每个像素包含三个颜色通道（RGB）。对于每个像素和每个颜色通道，我们都获得一个$\mathbb{R}$ 中的强度值。因此，一幅图像可以由一个元素 $z \in \mathbb{R}^{H \times W \times 3}$ 来表示。

- **视频**：视频本质上只是一系列随时间变化的图像。如果我们有 T 个时间点或帧，那么视频就可以表示为一个元素 $z \in \mathbb{R}^{T\times H\times W\times 3}$。

核心思想 1 · 对象即向量

我们将生成的对象视为向量 $z \in \mathbb{R}^d$。

1.2. 生成即采样

先来定义“生成”某个东西究竟是什么含义。举个例子：假设想要生成一张狗的图片。自然，存在大量能让我们满意的狗的图片。并不存在唯一一张“最优”的狗图片；取而代之的是一系列效果有好有坏的图片。

在机器学习中，通常把这种图片的多样性建模为**图片空间上的概率分布**。我们称该分布为**数据分布**，记作 p_{data}。从数学上看，可以把 p_{data} 理解成概率密度，也就是一个函数 $p_{\text{data}} : \mathbb{R}^d \to \mathbb{R}_{\geq 0}$，它为每一个可能的样本$z \in \mathbb{R}^d$ 分配一个似然值 $p_{\text{data}}(z) \geq 0$。

回到狗图片的例子：在这个分布下，越像狗的图片 z，对应的似然 $p_{\text{data}}(z)$ 就越高。于是，图片 / 视频 / 分子“效果好不好”这种主观性描述，就被替换成该样本在数据分布 p_{data} 下的**似然高低**。基于此，我们可以用数学语言把生成任务描述为：从（未知的）分布 p_{data} 中采样。

核心思想 2 · 生成即采样

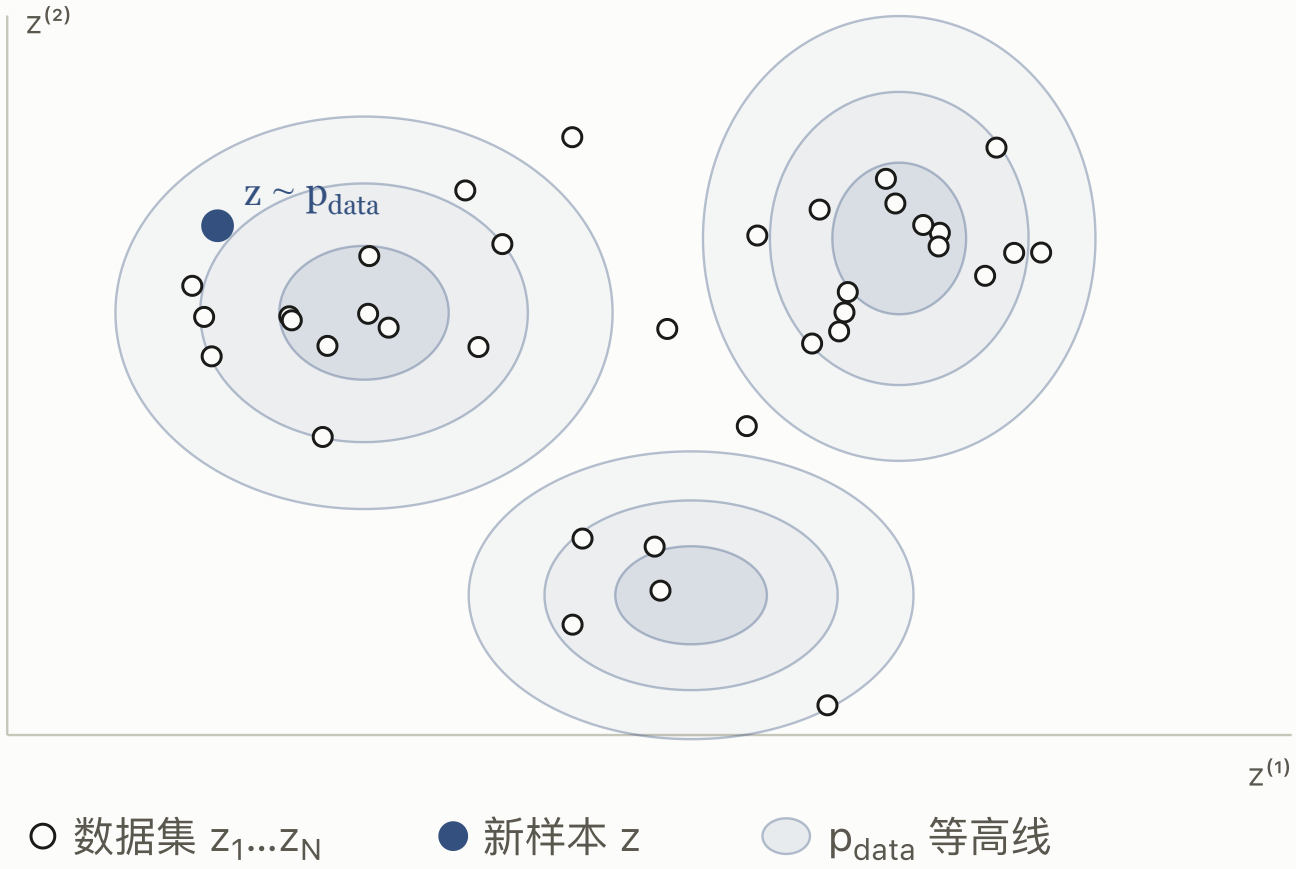
生成一个样本 z，建模为从数据分布中采样：$z \sim p_{\text{data}}$。

生成模型是一类机器学习模型，能够从 p_{data} 中生成样本。在机器学习领域，我们需要数据来训练模型。在生成建模中，通常假设可以获取有限个**独立采样自** p_{data} 的样本，这些样本共同作为真实分布的近似替代。

核心思想 3 · 数据集

数据集由有限个样本构成：$z_1, \ldots, z_N \sim p_{\text{data}}$。

对于图像任务，我们可以收集互联网上的公开图像来构建数据集；视频任务也可类似地使用 YouTube 数据。当数据集规模足够大时，它对底层真实分布 p_{data} 的表征能力会持续提升。



再采一个新样本　清空

已生成 1 个新样本（图中保留最近 8 个）—— 多点几次：它们总落在等高线最密的地方，即似然高的区域。

✦ 交互图 A · 数据分布（等高线）、数据集中的 N 个样本（空心点）与新采样的样本 z（实心点）。示意取 d = 2，真实图像空间的维度要高得多。

图中有三样东西值得分清：等高线代表我们永远看不见的真分布 p_{data}；空心点是手里仅有的数据集；实心点则是“生成”——它不是从数据集里挑一张，而是从同一个分布里重新抽一个样本，它通常并不在数据集里。

1.3. 引导式（条件）生成

在许多情况下，我们希望基于某些数据 y 生成对象。例如，我们可能希望基于 y = “一只狗在覆盖着雪的山丘上奔跑，背景是群山”来生成图像。我们可以将其重新表述为从条件分布中采样：

核心思想 4 · 引导式生成

引导式生成是从 $z \sim p_{\text{data}}(\cdot \mid y)$ 中采样，其中 y 为条件变量。我们称$p_{\text{data}}(\cdot \mid y)$ 为引导数据分布。引导式生成建模任务，通常要求模型能够基于任意选取的条件变量y 做条件生成，而非只能使用固定的 y。

沿用前文例子，我们也可以使用不同的文本提示作为条件，例如 y = “一张写实风格照片，画面里小猫正在吹生日蜡烛”。因此我们希望得到单个模型，能够接收任意这类条件 y。

无约束生成的相关技术可以很方便地推广到条件生成场景。因此，在前 3 个章节中，我们主要讨论**无条件生成**（同时记住：我们最终的目标是实现条件生成）。

1.4. 生成模型

抽象来讲，生成模型是一种算法，能够输出服从 $z \sim p_{\text{data}}$ 的样本（至少是近似服从该分布的样本）。如果 p_{data} 是狗图片的分布，那么该算法就会随机输出狗的图像。本课程将重点讲解基于流模型（flow）或扩散模型（diffusion）构建生成模型的具体方法，这两类模型代表了当前的前沿水平。不过需要记住，学界还发展出了许多其他类型的生成模型（未来或许还会诞生更多新模型）。

小结 2 · 生成即采样

1. 本文主要研究生成以向量 $z \in \mathbb{R}^d$ 形式表征的对象，例如图像、视频与分子结构。
2. 生成任务是从概率分布 p_{data} 中采样；训练阶段可以使用由样本$z_1, \dots, z_N \sim p_{\text{data}}$ 构成的数据集。
3. 引导式生成假定在标签 y 的条件下对分布建模，目标是从 $p_{\text{data}}(\cdot \mid y)$ 中采样；训练时可使用成对数据集 $(z_1, y), \dots, (z_N, y)$。
4. 我们的目标是构建生成模型：即经过训练后，能够输出服从 p_{data} 的样本的模型。

剩下的问题是：怎样的算法能把一个“容易采”的分布，变成这个复杂的 p_{data}？接下来两节分别给出两种答案——用常微分方程搭出 Flow 模型，用随机微分方程搭出 Diffusion 模型。

02. Flow 模型：用 ODE 生成

第一类生成模型建立在常微分方程之上。我们先定义轨迹、向量场与常微分方程，再引出“流”这一核心对象，然后讨论怎样在计算机上模拟它，最后把向量场交给神经网络，得到 Flow 模型。

2.1. ODE 与向量场

我们首先定义常微分方程（ODE）。ODE 的解由一条**轨迹**描述，即形如

$$X : [0,1] \to \mathbb{R}^d, \quad t \mapsto X_t$$

的函数，它将时间 t 映射到空间 $\mathbb{R}^d$ 中的某个位置。

每一个常微分方程都由一个**向量场** $\boldsymbol{u}$ 定义，向量场是如下函数：

$$\boldsymbol{u} : \mathbb{R}^d \times [0,1] \to \mathbb{R}^d, \quad (x,t) \mapsto \boldsymbol{u}_t(x)$$

也就是说，对任意时刻 t 和空间位置 x，都能得到向量 $\boldsymbol{u}_t(x) \in \mathbb{R}^d$，代表该点处的空间速度（见图 1）。

常微分方程对轨迹施加约束：我们希望轨迹 X 从初始点 x_0 出发，沿着向量场 $\boldsymbol{u}_t$ 的方向演化。该轨迹可以形式化写为下面方程的解：

$$\frac{d}{dt} X_t = \boldsymbol{u}_t(X_t) \qquad \blacktriangleright \text{常微分方程（ODE）} \tag{1a}$$

$$X_0 = x_0 \qquad \blacktriangleright \text{初值条件} \tag{1b}$$

式 (1a) 要求 X_t 的导数由向量场 $\boldsymbol{u}_t$ 给出的方向决定。式 (1b) 规定系统在$t = 0$ 时刻从 x_0 出发。

2.2. 流（Flow）

现在我们提出问题：若在 $t = 0$ 时 $X_0 = x_0$，那么在时刻 t，系统位于何处（即 X_t 是什么）？该问题由**流（flow）**函数回答，流是上述常微分方程的解：

$$\psi : \mathbb{R}^d \times [0,1] \to \mathbb{R}^d, \quad (x_0,t) \mapsto \psi_t(x_0) \tag{2a}$$

$$\frac{d}{dt} \psi_t(x_0) = \boldsymbol{u}_t\big(\psi_t(x_0)\big) \qquad \blacktriangleright \text{流的常微分方程} \tag{2b}$$

$$\psi_0(x_0) = x_0 \qquad \blacktriangleright \text{流的初值条件} \tag{2c}$$

给定初值 $X_0 = x_0$，ODE 的轨迹可以写成 $X_t = \psi_t(X_0)$。因此直观上，向量场、常微分方程与流是对同一事物的三种描述：向量场定义常微分方程，而方程的解就是流。

和所有方程一样，我们需要对常微分方程提出这样的问题：解是否存在，以及这个解是否唯一？数学中有一个基础结论：只要对 $\boldsymbol{u}_t$ 施加较弱的假设，**存在性与唯一性两者都成立！**

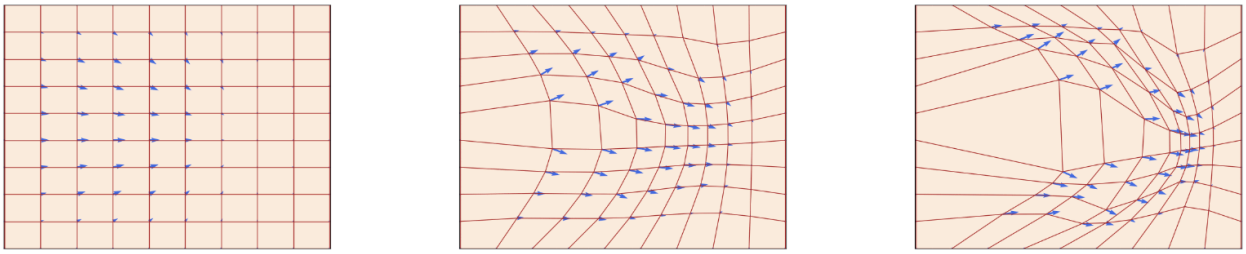


Figure 1: A flow $\psi_t : \mathbb{R}^d \to \mathbb{R}^d$ (red square grid) is defined by a velocity field $u_t : \mathbb{R}^d \to \mathbb{R}^d$ (visualized with blue arrows) that prescribes its instantaneous movements at all locations (here, $d = 2$). We show three different times t. As one can see, a flow is a diffeomorphism that "warps" space. Figure from [26].

图 1. 流 ψ_t（红色网格）由速度场 u_t（蓝色箭头）定义，图中展示三个时刻。流是一个"扭曲"空间的微分同胚（d = 2）。· 来源：MIT 6.S184 讲义，原图引自文献 [26]

定理 3 · 流的存在性与唯一性

若 $\boldsymbol{u} : \mathbb{R}^d \times [0,1] \to \mathbb{R}^d$ 连续可微，且导数有界，则式 (2) 中的常微分方程存在唯一解，由流 ψ_t 给出。此时对任意时刻 t，ψ_t 是微分同胚；即 ψ_t 连续可微，且其逆映射 ψ_t^{-1} 同样连续可微。

注意：流存在且唯一所需的假设，在机器学习场景中几乎总能满足。因为我们使用神经网络对 $\boldsymbol{u}_t(x)$ 做参数化，而神经网络的导数天然有界。因此，定理 3 对你而言无需担心，反而是个好消息：在我们关心的场景下，流一定存在，并且是常微分方程的唯一解。相关证明可参考文献 [32, 9]。

例 4 · 线性向量场

我们考察一个简单的向量场例子：$\boldsymbol{u}_t(x)$ 是关于 x 的线性函数，即$\boldsymbol{u}_t(x) = -\theta x$，其中 $\theta > 0$。此时函数

$$\psi_t(x_0) = \exp(-\theta t)\, x_0 \tag{3}$$

定义了满足方程 (2) 的流 ψ。你可以自行验证：首先 $\psi_0(x_0) = x_0$；再求导：

$$\begin{aligned}
\frac{d}{dt}\psi_t(x_0) &\overset{(3)}{=} \frac{d}{dt}\big(\exp(-\theta t)\, x_0\big) \\
&\overset{(i)}{=} -\theta \exp(-\theta t)\, x_0 \\
&\overset{(3)}{=} -\theta\, \psi_t(x_0) = \boldsymbol{u}_t\big(\psi_t(x_0)\big)
\end{aligned}$$

其中步骤 (i) 使用了链式法则。图 3 可视化了该形式的流，它会指数收敛到原点 0。

2.3. 常微分方程的数值模拟

一般而言，如果向量场 $\boldsymbol{u}_t$ 不像上一个例子那样简单，我们就无法写出流 ψ_t 的显式解析表达式。这种情况下，需要借助数值方法对常微分方程做模拟。所幸，这是数值分析中一个经典且研究充分的方向，已经存在大量高效算法 [21]。

欧拉法是最简单、最直观的方法之一。在欧拉法中，我们初始化 $X_0 = x_0$，然后按下式迭代更新：

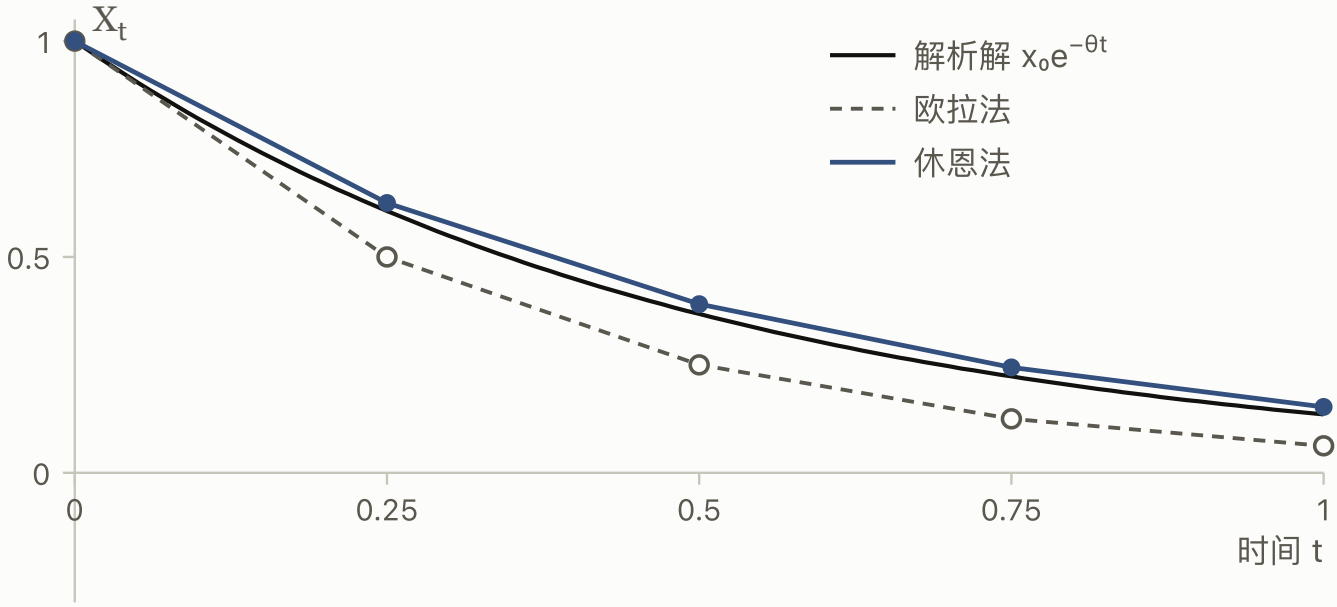
$$X_{t+h} = X_t + h\, \boldsymbol{u}_t(X_t) \qquad (t = 0, h, 2h, 3h, \ldots, 1-h) \tag{4}$$

其中步长 $h = n^{-1} > 0$，$n \in \mathbb{N}$ 代表模拟总步数。本课程范围内，欧拉法已经足够使用。

为让你感受更复杂的算法，我们来看**休恩法（Heun's method）**，其更新规则为：

$$\begin{aligned}
X'_{t+h} &= X_t + h\, \boldsymbol{u}_t(X_t) && \blacktriangleright \text{ 新状态的初步预估（与欧拉步一致）} \\
X_{t+h} &= X_t + \tfrac{h}{2}\big(\boldsymbol{u}_t(X_t) + \boldsymbol{u}_{t+h}(X'_{t+h})\big) && \blacktriangleright \text{ 用当前状态与预估状态的平均速度更新}
\end{aligned}$$

直观理解休恩法：先对下一步状态做初步预估 X'_{t+h}，再结合预估点的向量场修正最初的移动方向。



模拟步数 n = **4**（步长 h = 1/4）

t = 1 处的误差：欧拉法 0.0728 · 休恩法 0.0173 （欧拉法误差是休恩法的 4.2 倍）

✦ 交互图 B · 在例 4 的线性向量场 $u_t(x) = -\theta x$（$\theta = 2$，$x_0 = 1$）上比较两种数值方法。拖动滑块改变步数 n：步数越少，欧拉法偏离解析解越明显；休恩法在每个 n 下都更贴近，且误差随 n 增大缩小得快得多。

这正是例 4 的价值：解析解 $x_0e^{-\theta t}$ 给了我们一把“标准尺”。欧拉法每一步只看起点的速度，步长一大就会冲过头；休恩法多花一次向量场求值，用起点与预估点的平均速度修正方向，误差随步长缩小得快得多。

2.4. 流模型

现在我们可以基于常微分方程来构建生成模型：把向量场设计成神经网络参数化的向量场 $\boldsymbol{u}_{\theta,t}$。目前只需理解：$\boldsymbol{u}_{\theta,t}$ 是一个带参数 θ 的参数化函数$\boldsymbol{u}_{\theta,t} : \mathbb{R}^d \times [0,1] \to \mathbb{R}^d$。后续我们会讨论具体选用哪种神经网络结构。

回顾我们的目标：从分布 p_{data} 中采样得到样本 $z \sim p_{\text{data}}$，并且样本必须具有随机性。但注意，常微分方程本身是完全确定性的，不含随机因素。为引入随机性，我们只需让初值 X_0 服从随机分布。具体来说，选取一个初始分布 p_{init}。大多数场景下我们取 $p_{\text{init}} = \mathcal{N}(0, I_d)$，也就是标准高斯分布。最重要的一点：无论选择哪种初始分布，**必须保证推理阶段能够方便地从中采样**。

于是流模型由下面的常微分方程系统描述：

$$
\begin{aligned}
X_0 &\sim p_{\text{init}} && \blacktriangleright \text{随机初始化初值} \\
\frac{d}{dt} X_t &= \boldsymbol{u}_{\theta,t}(X_t) && \blacktriangleright \text{常微分方程}
\end{aligned}
$$

我们的目标：让轨迹的终点 X_1 服从数据分布 p_{data}，也就是

$$
X_1 \sim p_{\text{data}} \iff \psi_{\theta,1}(X_0) \sim p_{\text{data}}
$$

其中 $\psi_{\theta,t}$ 代表由向量场 $\boldsymbol{u}_{\theta,t}$ 诱导得到的流。

需要留意：虽然名字叫**流模型**，但神经网络参数化的对象是**向量场**，而不是直接参数化流本身。想要得到流，就必须对常微分方程做数值模拟。算法 1（Algorithm 1）总结了从流模型中采样的完整流程。

Algorithm 1 用欧拉法从流模型中采样（Sampling from a Flow Model with Euler method）

输入：神经网络向量场 u_t^θ，步数 n

1: 设 $t = 0$
2: 设步长 $h = \frac{1}{n}$
3: 采样 $X_0 \sim p_{\text{init}}$ ◂ 随机初始化！
4: **for** $i = 1, \dots, n-1$ **do**
5: $\quad X_{t+h} = X_t + h\, u_t^\theta(X_t)$
6: $\quad$ 更新 $t \leftarrow t + h$
7: **end for**
8: **return** X_1 ◂ 返回终点

✦ Algorithm 1 · 如何用流模型生成对象（据原讲义重排；右侧批注即原讲义中的红色批注）。

03. Diffusion 模型：用 SDE 生成

Flow 模型的轨迹一旦定下初值就完全确定。第二类生成模型在此基础上更进一步：让轨迹本身也带上随机性。为此我们需要随机微分方程，以及驱动它的基础随机过程——布朗运动。

3.1. 随机微分方程（SDE）

随机微分方程（Stochastic Differential Equation, SDE）在 ODE 的确定性轨迹基础上进行扩展，引入随机轨迹。随机轨迹通常被称为**随机过程** $(X_t)_{0\le t\le 1}$，其定义如下：对任意 $0 \le t \le 1$，X_t 是一个随机变量；

$$X : [0,1] \to \mathbb{R}^d, \quad t \mapsto X_t$$

每一次采样得到的 X，都对应一条随机轨迹。

特别地，当我们对同一个随机过程做两次模拟时，会得到不一样的结果，因为这套动力学本身就被设计为随机的。

3.2. 布朗运动（Brownian Motion）

SDE 借助布朗运动构建，布朗运动是一类基础随机过程，起源于对物理扩散过程的研究。你可以把布朗运动理解为**连续时间随机游走**。

下面给出定义：布朗运动 $W = (W_t)_{0\le t\le 1}$ 是满足下述条件的随机过程：$W_0 = 0$，轨迹 $t \mapsto W_t$ 连续，并且以下两条性质成立：

1. **正态增量**：对任意 $0 \le s < t$，有 $W_t - W_s \sim \mathcal{N}\big(0, (t-s) I_d\big)$。即增量服从高斯分布，方差随时间线性增长（$I_d$ 为单位矩阵）。
2. **独立增量**：对任意 $0 \le t_0 < t_1 < \cdots < t_n = 1$，增量$W_{t_1} - W_{t_0}, \ldots, W_{t_n} - W_{t_{n-1}}$ 为相互独立的随机变量。

布朗运动也称作维纳过程（Wiener process），这也是我们用符号 W 来表示它的原因。

我们可以取步长 $h > 0$，近似模拟布朗运动：初始化 $W_0 = 0$，按如下规则迭代更新：

$$W_{t+h} = W_t + \sqrt{h}\,\varepsilon_t, \quad \varepsilon_t \sim \mathcal{N}(0, I_d) \qquad (t = 0, h, 2h, \ldots, 1-h) \tag{5}$$

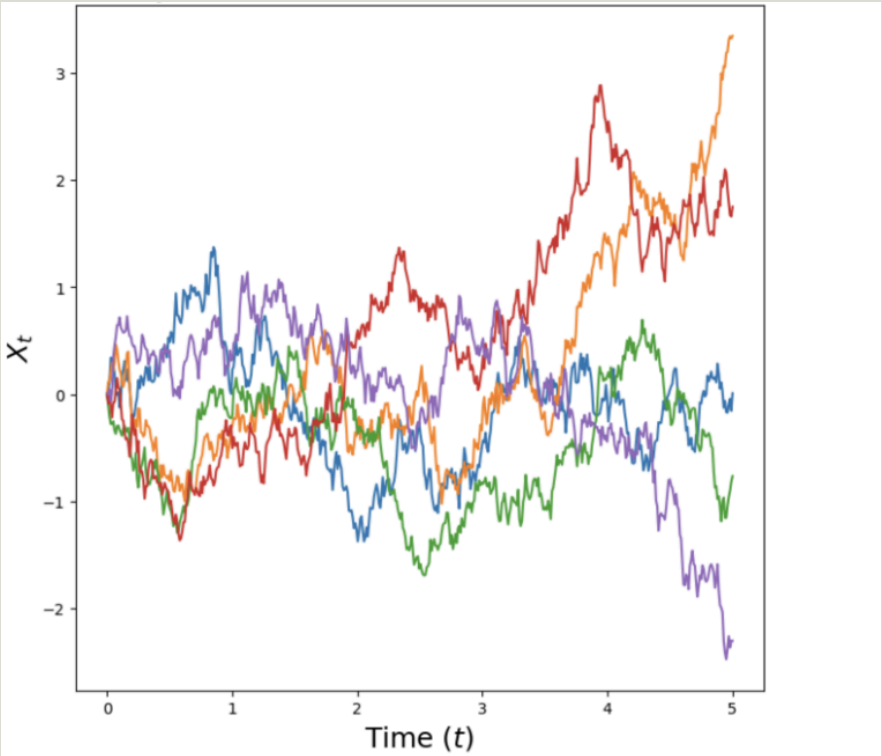


Figure 2: Sample trajectories of a Brownian motion W_t in dimension $d = 1$ simulated using Equation (5).

图 2. 用式 (5) 模拟的一维（d = 1）布朗运动 W_t 的样本轨迹。 · 来源： MIT 6.S184 讲义

图 2 绘制了几条布朗运动的示例轨迹。布朗运动在随机过程研究中的地位，等同于高斯分布在概率分布研究中的地位。从金融、统计物理到流行病学，布朗运动的应用范围远超机器学习。例如在金融领域，布朗运动被用于对复杂金融产品的价格建模。

仅从数学构造上看，布朗运动本身就十分奇妙：布朗运动的轨迹是连续的（你可以一笔画出，笔尖不用离开纸面），但轨迹长度却是无穷大（永远画不完）。

3.3. 从 ODE 到 SDE

SDE 的核心思想：在 ODE 的确定性动力学基础上，增加由布朗运动驱动的随机动力学。由于全部过程是随机的，我们无法再像式 (1a) 那样直接定义导数。因此需要找到一种**不依赖导数**、但和 ODE 等价的表达形式。

为此，我们将 ODE 的轨迹 $(X_t)_{0\le t\le 1}$ 改写如下：

$$
\begin{aligned}
& \frac{d}{dt} X_t = u_t(X_t) && \blacktriangleright \text{导数形式} \\
\iff\ & \frac{1}{h}\big(X_{t+h} - X_t\big) = u_t(X_t) + R_t(h) && \blacktriangleright \text{仅使用导数的定义 (i)} \\
\iff\ & X_{t+h} = X_t + h\, u_t(X_t) + h\, R_t(h) && \blacktriangleright \text{无穷小更新形式}
\end{aligned}
$$

其中 $R_t(h)$ 是当 h 很小时可以忽略的余项，满足 $\lim_{h\to 0} R_t(h) = 0$。

上面的推导只是重申已知结论：ODE 的轨迹 $(X_t)_{0\le t\le 1}$ 在每一个时间步，都会沿着 $u_t(X_t)$ 的方向前进一小步。现在我们修改这个式子，将其变为随机版本：SDE 的轨迹 $(X_t)_{0\le t\le 1}$ 在每一个时间步，除了沿着 $u_t(X_t)$ 的方向前进一小步，还会叠加布朗运动带来的随机增量：

$$
X_{t+h} = \underbrace{X_t + h\, u_t(X_t)}_{\text{确定性项}} + \underbrace{\sigma_t\,(W_{t+h} - W_t)}_{\text{随机项}} + \underbrace{h\, R_t(h)}_{\text{误差项}} \tag{6}
$$

式中 $\sigma_t \ge 0$ 为**扩散系数**；$R_t(h)$ 是随机误差项，满足当 $h \to 0$ 时，标准差 $\mathbb{E}\big[\|R_t(h)\|^2\big]^{1/2} \to 0$。

上面这个式子就是随机微分方程（SDE）。通常使用下面这种符号简写形式：

$$
dX_t = u_t(X_t)\, dt + \sigma_t\, dW_t \qquad \blacktriangleright \text{SDE} \tag{7a}
$$

$$
X_0 = x_0 \qquad \blacktriangleright \text{初值条件} \tag{7b}
$$

但务必记住：符号 dX_t 只是式 (6) 的**形式化简写，并非严格数学定义**。

遗憾的是，SDE 不再存在类似 ODE 的流映射 ψ_t。原因在于：演化过程本身带有随机性，X_t 不再完全由初值$X_0 \sim p_{\text{init}}$ 唯一确定。不过，和 ODE 类似，我们仍然有解的存在唯一性：

定理 5 · SDE 解的存在唯一性

若 $\boldsymbol{u} : \mathbb{R}^d \times [0,1] \to \mathbb{R}^d$ 连续可微且导数有界，同时 σ_t 连续，则式 (7) 中的 SDE 存在唯一随机过程 $(X_t)_{0\le t\le 1}$，满足式 (6)。

如果这是随机分析课程，我们会用好几节课来证明这个定理，并以严格的数学方式构建 SDE：从基本原理出发构造布朗运动，再通过随机积分构造随机过程 X_t。但本课程聚焦机器学习，如需更严谨的推导可以参考文献 [29]。

最后注意：**每一个 ODE 都可以看作 SDE**，只需要令扩散系数恒等于 0，即 $\sigma_t = 0$。因此在本课程后续内容中，我们把 ODE 视作 SDE 的一种特例。

3.4. 奥恩斯坦-乌伦贝克过程

例 6 · 奥恩斯坦-乌伦贝克过程（Ornstein-Uhlenbeck，OU 过程）

我们取恒定扩散系数 $\sigma_t = \sigma \geq 0$，恒定线性漂移项 $\boldsymbol{u}_t(x) = -\theta x$（$\theta > 0$），由此得到 SDE：

$$dX_t = -\theta X_t \, dt + \sigma \, dW_t \tag{8}$$

该 SDE 的解 $(X_t)_{0 \leq t \leq 1}$ 称为**奥恩斯坦-乌伦贝克（OU）过程**，其可视化见图 3。向量场 $-\theta x$ 会把过程拉回中心点 0（漂移方向永远与当前位置相反）；而扩散系数 σ 持续引入噪声。当模拟时间 $t \to \infty$ 时，该过程会收敛到高斯分布 $\mathcal{N}\left(0, \sigma^2/(2\theta)\right)$。
注意：当 $\sigma = 0$ 时，就退化为我们之前式 (3) 研究过的、具有线性向量场的流。

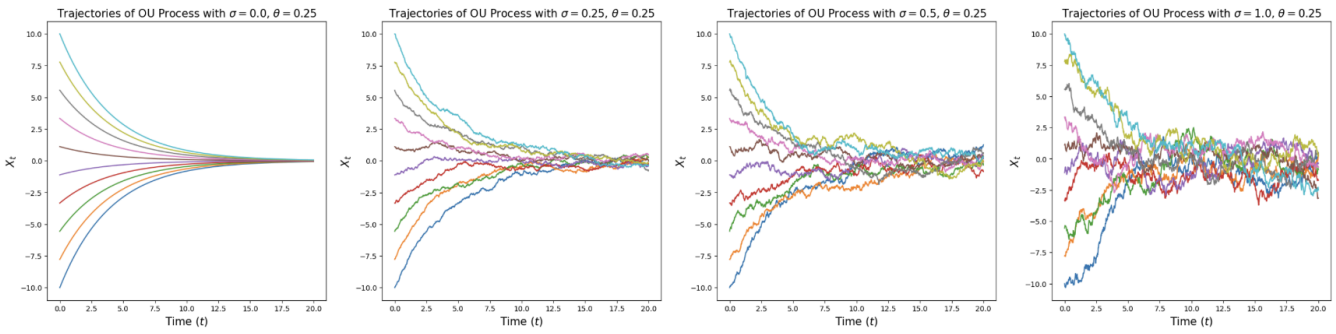
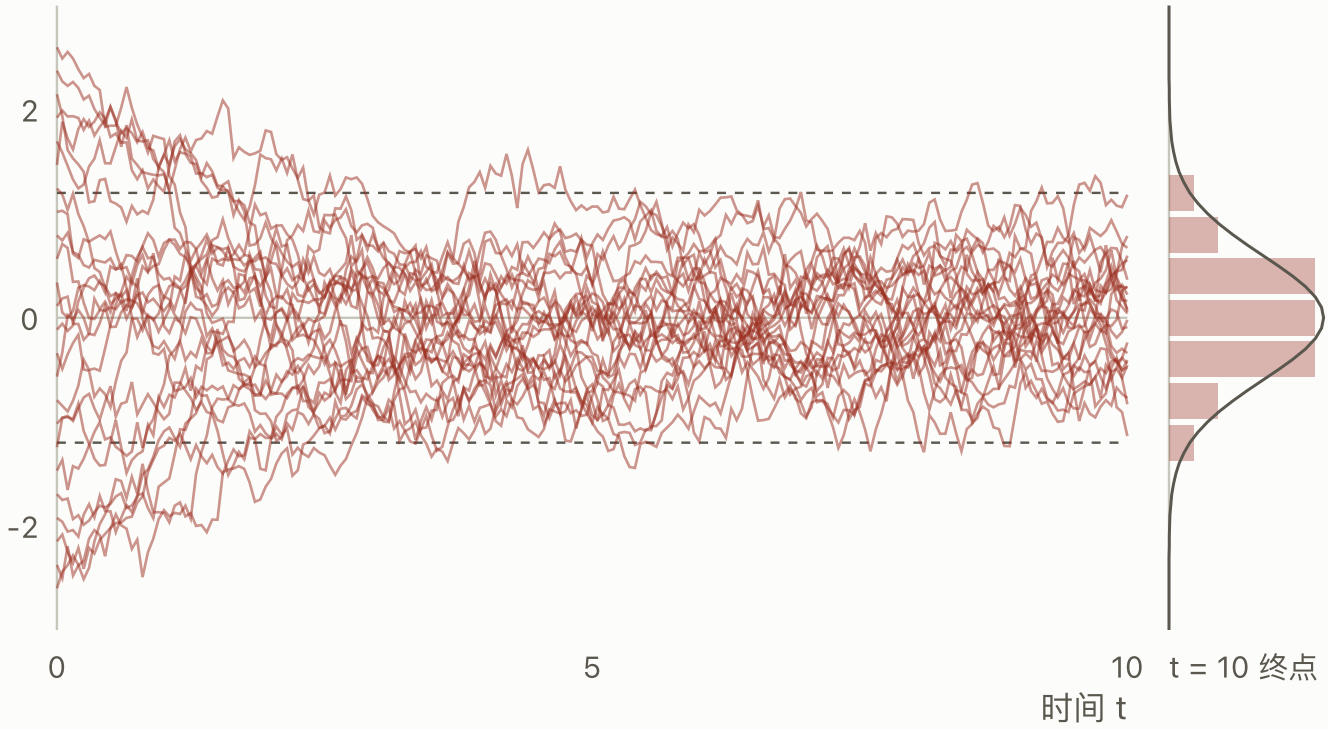


Figure 3: Illustration of Ornstein-Uhlenbeck processes (Equation (8)) in dimension $d = 1$ for $\theta = 0.25$ and various choices of σ (increasing from left to right). For $\sigma = 0$, we recover a flow (smooth, deterministic trajectories) that converges to the origin as $t \to \infty$. For $\sigma > 0$ we have random paths which converge towards the Gaussian $\mathcal{N}(0, \frac{\sigma^2}{2\theta})$ as $t \to \infty$.

图 3. 一维（d = 1）OU 过程（式 8），θ = 0.25，σ 从左到右取 0、0.25、0.5、1.0。σ = 0 时得到随 t → ∞ 收敛到原点的流（平滑、确定性的轨迹）；σ > 0 时是随 t → ∞ 收敛到高斯分布 N(0, σ²/(2θ)) 的随机路径。 · 来源：MIT 6.S184 讲义

图 3 只给了四个固定的 σ。把 σ 做成一个连续的旋钮，能更直接地看到“ODE 是 σ = 0 的 SDE”这句话的含义：



扩散系数 σ = **0.60**

换一组噪声

σ = 0.60：漂移项把轨迹拉回 0，噪声持续把它推开；终点散布趋近 N(0, σ²/(2θ))，标准差 ≈ 0.60（虚线为 ±2 倍标准差）。

✦ 交互图 C · OU 过程（24 条轨迹，用 3.5 节式 (9) 的欧拉-丸山法模拟）。为了在 t ≤ 10 内看清收敛，这里取 θ = 0.5（比图 3 的 0.25 大），时间轴也延长到 10。拖动 σ：同一组噪声只被 σ 缩放，σ = 0 时轨迹变回例 4 的平滑流；右侧是 t = 10 时终点的直方图与理论密度 N(0, σ²/(2θ))。

3.5. SDE 的数值模拟

我们可以从一个更直观的角度理解 SDE：**如何对 SDE 做数值模拟?**

最简单的模拟方案称为欧拉-丸山法（**Euler-Maruyama method**）。对于 SDE 而言，欧拉-丸山法的地位，就如同欧拉法之于 ODE。

使用欧拉-丸山法时，初始化 $X_0 = x_0$，按下式迭代更新：

$$X_{t+h} = X_t + h\,u_t(X_t) + \sqrt{h}\,\sigma_t\,\varepsilon_t, \quad \varepsilon_t \sim \mathcal{N}(0, I_d) \tag{9}$$

其中 $h = n^{-1} > 0$ 是步长超参数，$n \in \mathbb{N}$。换言之，采用欧拉-丸山法做模拟时，我们沿着 $u_t(X_t)$的方向前进一小步，同时叠加一份由 $\sqrt{h}\,\sigma_t$ 缩放的高斯噪声。

本课程在模拟 SDE 时（包括配套实验），一般都会使用欧拉-丸山法。

3.6. 扩散模型

现在我们可以仿照 ODE 构建流模型的思路，基于 SDE 构建生成模型。回顾目标：把简单分布 p_{init} 转化为复杂的数据分布p_{data}。和 ODE 的情况类似，用 $X_0 \sim p_{\text{init}}$ 做随机初始化，再对 SDE 进行数值模拟，是实现该变换的自然方案。

为参数化这个 SDE，我们只需用神经网络 $\boldsymbol{u}_{\theta,t}$ 对其核心组分——向量场 $\boldsymbol{u}_t$ 做参数化。由此，扩散模型可写为：

$$
\begin{aligned}
X_0 &\sim p_{\text{init}} && \blacktriangleright \text{随机初始化} \\
dX_t &= \boldsymbol{u}_{\theta,t}(X_t)\,dt + \sigma_t\,dW_t && \blacktriangleright \text{SDE}
\end{aligned}
$$

算法 2 描述了使用欧拉-丸山法从扩散模型中采样的流程。

Algorithm 2 从扩散模型中采样（Sampling from a Diffusion Model, Euler-Maruyama method）

输入：神经网络 u_t^θ，步数 n，扩散系数 σ_t

1: 设 $t = 0$
2: 设步长 $h = \frac{1}{n}$
3: 采样 $X_0 \sim p_{\text{init}}$
4: **for** $i = 1, \dots, n$ **do**
5: 　　采样 $\epsilon \sim \mathcal{N}(0, I_d)$
6: 　　$X_{t+h} = X_t + h\,u_t^\theta(X_t) + \sigma_t \sqrt{h}\,\epsilon$
7: 　　更新 $t \leftarrow t + h$
8: **end for**
9: **return** X_1

✦ Algorithm 2 · 用欧拉-丸山法从扩散模型中采样（据原讲义重排）。与 Algorithm 1 相比，多了第 5 行的噪声采样和第 6 行的随机项；另外原讲义中两份算法的循环上界分别写作 n – 1 与 n。

本节内容总结如下：

总结 7 · SDE 生成模型

在本文档中，扩散模型由两部分组成：带参数 θ、用于参数化向量场的神经网络 $\boldsymbol{u}_{\theta,t}$，以及固定的扩散系数 σ_t：

- 神经网络：$\boldsymbol{u}_\theta : \mathbb{R}^d \times [0,1] \to \mathbb{R}^d,\ (x,t) \mapsto \boldsymbol{u}_{\theta,t}(x)$，参数为 θ
- 固定项：$\sigma_t : [0,1] \to [0,\infty),\ t \mapsto \sigma_t$

从 SDE 模型中采样（即生成样本）的流程如下：

1. 初始化：$X_0 \sim p_{\text{init}}$ ▶ 从简单分布（例如高斯分布）初始化
2. 模拟：$dX_t = \boldsymbol{u}_{\theta,t}(X_t)\,dt + \sigma_t\,dW_t$ ▶ 从 $t=0$ 到 $t=1$ 模拟 SDE
3. 目标：$X_1 \sim p_{\text{data}}$ ▶ 目标是让 X_1 服从数据分布 p_{data}

当扩散模型取 $\sigma_t = 0$ 时，该模型就退化为流模型。

结语：Flow 与 Diffusion 对照

回头看，两类模型做的是同一件事：从易采样的 p_{init} 出发，沿着神经网络给出的向量场走到 p_{data}。它们的根本区别，是走路时要不要再掷骰子。下表把正文中散落的事实并排放在一起：

表 1. Flow 模型与 Diffusion 模型对照

维度	FLOW 模型	DIFFUSION 模型
动力学	ODE：$\frac{d}{dt}X_t = \boldsymbol{u}_{\theta,t}(X_t)$	SDE：$dX_t = \boldsymbol{u}_{\theta,t}(X_t)\,dt + \sigma_t\,dW_t$
随机性来源	只有初值 $X_0 \sim p_{\text{init}}$	初值 + 每一步的布朗运动增量
流映射 ψ_t	存在且唯一，且为微分同胚（定理 3）	不存在；但解作为随机过程存在且唯一（定理 5）
神经网络参数化	向量场 $\boldsymbol{u}_{\theta,t}$	向量场 $\boldsymbol{u}_{\theta,t}$；扩散系数 σ_t 固定
数值方法	欧拉法（式 4）、休恩法	欧拉-丸山法（式 9）
单步更新	$X_{t+h} = X_t + h\,\boldsymbol{u}_{\theta,t}(X_t)$	$X_{t+h} = X_t + h\,\boldsymbol{u}_{\theta,t}(X_t) + \sqrt{h}\,\sigma_t\,\varepsilon_t$
采样算法	Algorithm 1	Algorithm 2
两者关系	扩散模型取 $\sigma_t = 0$ 的特例	一般情形

从表中可以看出：在本讲的表述里，两类模型共用同一个神经网络向量场，真正把它们区分开的只有扩散系数 σ_t 这一个旋钮。